

Agent Phantom Recovery

17-full-implementation-plan.md

Full Production Implementation Plan

Version: 1.0

Status: Master Implementation Roadmap

1. Purpose

This document serves as the master implementation roadmap for Agent Phantom Recovery.

While previous documents define:

Vision
Architecture
Database
APIs
LLM Integration
RAG
Modules

this document answers:

How do we actually build the entire system?

This roadmap combines every previous document into a single implementation strategy.

2. Project Goal

Agent Phantom Recovery is not a chatbot.

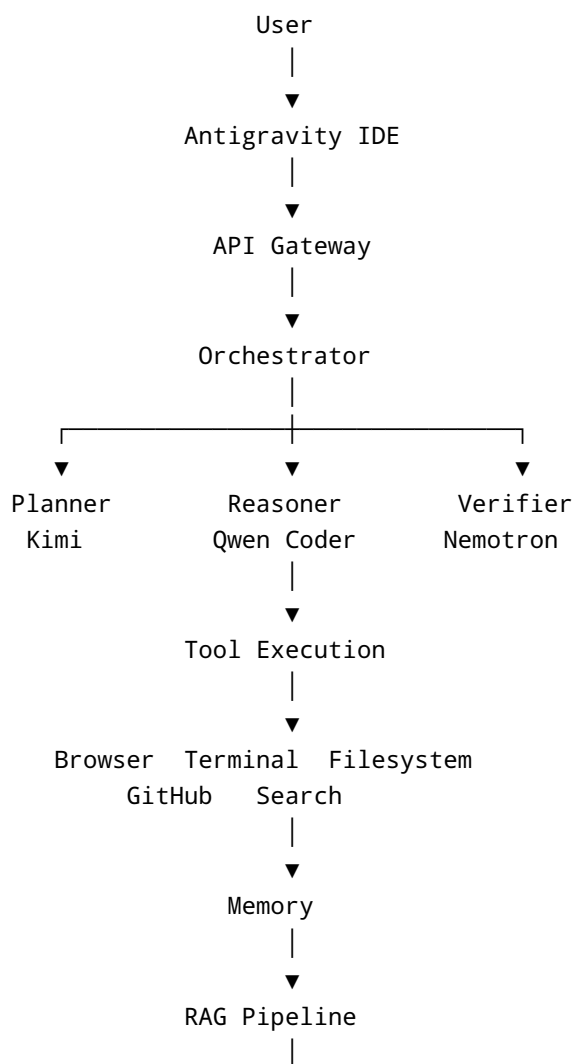
The objective is to build a long-horizon autonomous engineering system capable of:

Goal Understanding
↓
Planning
↓

Tool Usage
↓
Observation
↓
Reasoning
↓
Verification
↓
Execution
↓
Recovery
↓
Completion

The system should be able to perform extended engineering workflows with minimal intervention.

3. Final System Architecture



4. Build Strategy

The system should be developed in progressive layers.

Never build advanced features before foundational systems exist.

Development Sequence

```
Foundation
↓
Backend
↓
LLMs
↓
Tools
↓
Memory
↓
Repository Intelligence
↓
Execution Engine
↓
Antigravity IDE
↓
Production Hardening
```

5. Technology Stack

Frontend

```
Next.js
TypeScript
Tailwind CSS
shadcn/ui
React Query
Zustand
Monaco Editor
```

Backend

FastAPI
Python
Pydantic
SQLAlchemy

Database

Supabase PostgreSQL
pgvector
Redis

Storage

Supabase Storage

LLM Layer

OpenRouter

Models

Planner
↓
Kimi K2.6

Reasoner
↓
Qwen3 Coder 480B

Verifier
↓
Nemotron 3 Ultra

6. Phase 0 — Foundation

Duration:

Week 1

Deliverables

Monorepo

```
apps/  
packages/  
services/
```

Environment Management

```
.env  
.env.local  
.env.production
```

CI/CD

GitHub Actions

Supabase Setup

```
Database  
Storage  
Auth  
Realtime
```

Completion Criteria

```
Project builds successfully  
Deployment pipeline works
```

7. Phase 1 — Authentication & User Management

Duration:

Week 1

Features

Registration
Login
Logout
Session Management
Profiles

APIs

/auth/*

Database Tables

profiles
user_sessions

Completion Criteria

Protected APIs functional
User sessions stable

8. Phase 2 — Project & Task Management

Duration:

Week 2

Project Module

Features:

Create Project
Update Project
Delete Project

Task Module

Features:

Create Task
Track Status
Pause
Resume
Cancel

State Machine

CREATED
PLANNING
ANALYZING
EXECUTING
VERIFYING
COMPLETED
FAILED

Completion Criteria

Task lifecycle fully operational

9. Phase 3 — LLM Infrastructure

Duration:

Week 2-3

LLM Gateway

Build:

Routing Layer
Rate Limits
Retries
Fallbacks
Monitoring

Planner Service

Model:

Kimi

Capabilities:

Goal Analysis
Task Planning
Replanning

Reasoner Service

Model:

Qwen

Capabilities:

Repository Analysis
Root Cause Analysis
Patch Generation

Verifier Service

Model:

Nemotron

Capabilities:

Review
Validation
Regression Detection

Completion Criteria

Three-agent pipeline operational

10. Phase 4 — Tool Infrastructure

Duration:

Week 3-4

Tool Registry

Build:

Registration
Discovery
Health Checks

Execution Engine

Build:

Tool Runner
Sandbox
Timeouts
Retries

Terminal Tool

Capabilities:

Commands
Streaming
Processes

Browser Tool

Capabilities:

Navigation
DOM Access
Screenshots
Automation

Filesystem Tool

Capabilities:

Read
Write
Move
Search

Git Tool

Capabilities:

Commit
Diff
Branch
Status

Completion Criteria

All tools executable
Streaming operational

11. Phase 5 — Memory System

Duration:

Week 4

Working Memory

Stores:

Active Task Context

Session Memory

Stores:

Current Session Knowledge

Project Memory

Stores:

Architecture Knowledge
Repository Knowledge

Experience Memory

Stores:

Successful Fixes
Failed Fixes
Patterns

Completion Criteria

Memory retrieval operational

12. Phase 6 — RAG Infrastructure

Duration:

Week 5

Repository Ingestion

Pipeline:

Clone
↓
Parse
↓
Chunk
↓
Embed

↓
Store

Vector Search

Implement:

Semantic Search
Similarity Search

Context Builder

Implement:

Ranking
Compression
Assembly

Completion Criteria

Repository retrieval operational

13. Phase 7 — Repository Intelligence

Duration:

Week 5-6

Symbol Extraction

Generate:

Functions
Classes
Methods

Dependency Analysis

Generate:

Dependency Graph

Call Graph Analysis

Generate:

Execution Relationships

Architecture Analysis

Generate:

System Summaries

Completion Criteria

Large repositories searchable

14. Phase 8 — Orchestrator

Duration:

Week 6

Responsibilities

Task Lifecycle
Agent Coordination
Execution State
Recovery

Core Loop

Goal
↓
Plan
↓
Tool
↓
Observe
↓
Replan
↓
Continue

Completion Criteria

Autonomous task execution works

15. Phase 9 — Checkpoint & Recovery System

Duration:

Week 6

Features

Snapshots
Resume
Recovery

Goal

Enable:

Multi-Hour Tasks

Completion Criteria

Recovery from crashes works

16. Phase 10 — Verification Layer

Duration:

Week 7

Verification Pipeline

Reasoner
↓
Verifier
↓
Approve/Reject

Verification Types

Evidence Validation
Patch Review
Regression Detection

Completion Criteria

Independent verification operational

17. Phase 11 — Antigravity IDE

Duration:

Week 7-8

Workspace 1

Chat & Agent Console

Features:

Task Creation
Agent Status
Plans

Workspace 2

Source Code Workspace

Features:

Monaco Editor
Search
Diffs

Workspace 3

Terminal Workspace

Features:

Streaming Commands
Live Output

Workspace 4

Browser Workspace

Features:

Live Browser
Agent Actions

Workspace 5

Memory Workspace

Features:

Retrieved Context
Memory Inspection

Workspace 6

Timeline Workspace

Features:

Events
Execution Tracking

Completion Criteria

All workspaces integrated

18. Phase 12 — Realtime Infrastructure

Duration:

Week 8

Build

WebSockets
Realtime Streams
Event Channels

Streams

Tasks
Terminal
Browser
Memory
Agent Updates

Completion Criteria

Realtime synchronization operational

19. Phase 13 — GitHub Integration

Duration:

Week 8

Features

Repository Sync
Commit Creation

Pull Requests
Reviews

Completion Criteria

PR workflow functional

20. Phase 14 — Observability

Duration:

Week 9

Metrics

Track:

Latency
Success Rate
Token Usage

Logging

Track:

Agent Actions
Tool Usage
Errors

Tracing

Track:

Task Lifecycles

Completion Criteria

Full visibility achieved

21. Phase 15 — Security Layer

Duration:

Week 9

Add

RLS Policies
Audit Logs
Rate Limits
Secret Detection

Security Goal

Prevent:

Unauthorized Access
Credential Exposure
Unsafe Execution

22. Phase 16 — Performance Optimization

Duration:

Week 10

Optimize

Retrieval
Tool Execution
Caching
Queries

Performance Targets

Retrieval < 1 sec
Memory < 500ms
Task Updates < 100ms

23. Phase 17 — Production Deployment

Duration:

Week 10

Infrastructure

Frontend Hosting
Backend Hosting
Supabase
Redis
Monitoring

Deployment Pipeline

Development
↓
Staging
↓
Production

Completion Criteria

Production release ready

24. MVP Definition

A successful MVP must support:

```
Create Task
↓
Generate Plan
↓
Use Tools
↓
Analyze Repository
↓
Generate Fix
↓
Verify Fix
↓
Display Results
```

Everything else is secondary.

25. Version Roadmap

Version 1.0

```
Planner
Reasoner
Verifier
Tools
Memory
RAG
Antigravity IDE
```

Version 1.5

Advanced Browser Automation
Better Recovery
Improved Verification

Version 2.0

Multi-Agent Expansion
Knowledge Graph
Advanced Memory Networks

26. Critical Success Metrics

Engineering Capability

Repository Understanding
Patch Accuracy
Verification Accuracy

Execution Capability

Task Completion Rate
Recovery Rate
Tool Success Rate

User Experience

Workspace Responsiveness
Realtime Performance
Execution Transparency

27. Final System Build Order

```
Foundation
↓
Authentication
↓
Projects
↓
Tasks
↓
LLM Layer
↓
Tools
↓
Memory
↓
RAG
↓
Repository Intelligence
↓
Orchestrator
↓
Verification
↓
Antigravity IDE
↓
Realtime
↓
GitHub
↓
Observability
↓
Security
↓
Optimization
↓
Production
```

28. Final Summary

Agent Phantom Recovery is implemented as a layered autonomous engineering platform.

The implementation strategy intentionally prioritizes:

```
Planning
↓
Reasoning
↓
Verification
↓
Execution
↓
Recovery
```

over rapid feature accumulation.

Following this roadmap results in a production-grade system capable of long-horizon engineering workflows, repository analysis, tool orchestration, autonomous investigation, patch generation, validation, and execution inside the Antigravity IDE environment.

End of Document

Status: Approved

Version: 1.0

Document Type: Full Production Implementation Plan